SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

CO5 – MOCK INTERVIEW QUESTIONS

Student Name: BATHALA HEMANTH KUMAR

Register Number: 192324238

Question 1

You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Answer

For a hospital patient-data system, I would use Pandas for structured data processing and NumPy for numerical operations. The data would be read from CSV files, cleaned, standardized, validated, analyzed, and finally saved as a processed dataset.

1. Reading the CSV File

Pandas can load the continuously collected CSV data into a DataFrame.

2. Inspecting the Dataset

First, I would inspect the data types, missing values, and duplicate records.

3. Handling Missing Values

Missing values can be handled according to the importance of each column. For numerical columns, missing values can be replaced using the median or mean. For categorical columns, the most frequently occurring value can be used. Records with too many missing fields may be removed when they cannot be reliably used.

4. Removing Duplicate Entries

Duplicate patient records can produce incorrect reports. Pandas can identify and remove duplicates. If a unique Patient_ID is available, duplicates can specifically be identified using that column.

5. Standardizing Inconsistent Units

Different records may use different units. A standard unit should be selected. For example, temperature can be converted from Fahrenheit to Celsius and weight from pounds to kilograms.

6. Data Type Conversion

Correct data types improve processing efficiency and prevent calculation errors. Numeric fields can be converted using pd.to_numeric(), and dates can be converted using pd.to_datetime(). Invalid values can be coerced to missing values and handled separately.

7. Filtering Data

Pandas can select records satisfying specific conditions. For example, patients with temperature above 38°C can be filtered. Multiple conditions can also be combined, such as high temperature and high heart rate.

8. Indexing

Indexing allows specific rows and columns to be accessed efficiently. loc can select records using labels or conditions, iloc can select records by position, and Patient_ID can be set as an index for patient-specific retrieval.

9. Using NumPy for Numerical Processing

NumPy can efficiently perform numerical calculations on patient measurements, including mathematical operations, conditional processing, and array-based calculations.

10. Aggregation

Aggregation helps generate hospital reports such as average temperature, average age, and patient counts. Pandas groupby() and agg() can summarize these measures by diagnosis or another category.

11. Sorting

Records can be sorted according to important measurements, such as temperature, so high-risk patients can be identified quickly.

12. Handling Unexpected Data Anomalies

Unexpected anomalies should be detected before generating reports. Examples include negative age or weight, impossible temperature values, invalid patient IDs, incorrect dates, extreme measurements, unknown categories, and unexpected units. Suspicious values should be corrected when possible or flagged for manual review.

13. Saving the Processed Data

After cleaning and standardization, the processed DataFrame can be saved to a new CSV file. The original raw data should be preserved separately.

14. Near Real-Time Processing Strategy

For near real-time reporting, I would process newly received CSV records in batches rather than repeatedly processing the entire historical dataset. The workflow would be: CSV Input → Data Validation → Missing Value Handling → Duplicate Removal → Unit Standardization → Anomaly Detection → Filtering → Aggregation → Reporting → Processed CSV.

Conclusion

NumPy and Pandas provide an efficient solution for hospital data processing. Pandas can manage CSV files, DataFrames, missing values, duplicates, indexing, filtering, grouping, sorting, and file I/O, while NumPy provides efficient numerical calculations. Validation and anomaly detection are essential because hospital reports must be based on reliable and standardized data.

Important Code Examples

```
import pandas as pd

import numpy as np

df = pd.read_csv("patient_data.csv")

print(df.info())

print(df.isnull().sum())

df["Age"] = df["Age"].fillna(df["Age"].median())

df = df.drop_duplicates()

df = df.sort_values(by="Temperature", ascending=False)

fever_patients = df[df["Temperature"] > 38]

report = df.groupby("Diagnosis").agg(Patient_Count=("Patient_ID", "count"))

df.to_csv("processed_patient_data.csv", index=False)
```

Question 2

You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using NumPy and Pandas? Explain the steps briefly.

Answer

Before training a machine learning model, the dataset must be converted into a clean numerical format. The main preprocessing steps are data inspection, handling missing values, encoding categorical variables, and normalizing numerical features.

1. Load the Dataset

The dataset can be loaded using Pandas and its first few records and data types should be inspected.

2. Identify Numerical and Categorical Features

Numerical columns contain values such as age, salary, height, or marks, while categorical columns contain values such as gender, city, or department. Pandas can separate these columns using select_dtypes().

3. Handle Missing Numerical Values

Missing numerical values can be replaced using the mean or median. The median is often useful because it is less affected by extreme values.

4. Handle Missing Categorical Values

Missing categorical values can be replaced with the mode. Alternatively, missing categories can be represented using a separate value such as Unknown.

5. Encode Categorical Variables

Machine learning algorithms generally require numerical input. Therefore, categorical values need to be converted into numerical representations. One-hot encoding can be performed with pd.get_dummies().

6. Label Encoding for Ordered Categories

If a categorical feature has an actual order, numerical mapping can be used. For example, Low → 0, Medium → 1, and High → 2.

7. Normalize Numerical Features

Numerical features may have very different scales. Min-Max normalization transforms values approximately into the range 0 to 1 and prevents large-scale features from dominating certain machine learning algorithms.

8. Standardization

Another option is standardization using the mean and standard deviation. This produces features with approximately mean 0 and standard deviation 1.

9. Check the Processed Dataset

After preprocessing, the dataset should be checked again to confirm that missing values have been handled and numerical features have been transformed correctly.

10. Save the Preprocessed Dataset

The final dataset can be saved for use by the machine learning model using to_csv().

Preprocessing Workflow

Raw Dataset → Load using Pandas → Identify Numerical and Categorical Features → Handle Missing Values → Encode Categorical Variables → Normalize/Standardize Numerical Features → Validate Processed Data → Save Dataset → Use for Machine Learning Model.

Conclusion

NumPy and Pandas provide the basic tools required for preparing a dataset for machine learning. Pandas is useful for loading, inspecting, cleaning, encoding, and saving data, while NumPy supports efficient numerical operations. Proper preprocessing improves data quality and makes the dataset suitable for machine learning algorithms.

Important Code Examples

```
import pandas as pd

import numpy as np
```

```
df = pd.read_csv("dataset.csv")

numerical = df.select_dtypes(include=np.number).columns

categorical = df.select_dtypes(exclude=np.number).columns

for col in numerical:

    df[col] = df[col].fillna(df[col].median())

for col in categorical:

    df[col] = df[col].fillna(df[col].mode()[0])

df = pd.get_dummies(df, columns=["Gender"], dtype=int)

df["Risk"] = df["Risk"].map({"Low": 0, "Medium": 1, "High": 2})

df.to_csv("preprocessed_dataset.csv", index=False)
```